

마을 사보타주 추리 게임 AI(LLM) 활용 내역

Anthropic Claude API 기반 실시간 콘텐츠 생성 파이프라인

해커톤 : AI_Hakathon_NAN_2026

문서 작성 : 류수정 (기획)

작성일 : 2026-08-08

1. 개요

이 게임은 사보타주 사건·단서·NPC 반응 등 핵심 콘텐츠 대부분을 정적으로 미리 써두지 않고, **플레이 도중 LLM이 실시간으로 생성하도록** 설계했다. 그 결과 같은 범인이라도 매 판 단서 문구·대화·엔딩이 조금씩 달라지고, 반복 플레이의 지루함이 줄어든다. LLM 호출은 전용 서비스인 `llm-proxy`가 전담하고, 게임 서버(`backend`)는 내부 API 키로 인증된 요청만 이 서비스를 호출한다.

2. 사용한 AI

구분	도구 / 모델	용도
콘텐츠 생성 (런타임)	Anthropic Claude API Claude Opus 5 Claude Sonnet 5	게임 내 대화·단서·이벤트·엔딩 등 8종 콘텐츠 실시간 생성
코드 개발 보조	(팀 확인 필요)	커밋 브랜치명에 <code>codex/village-map-implementation</code> 이 존재해 Codex 계열 AI 코딩 도구를 사용한 것으로 추정되나, 정확한 도구명·활용 범위는 팀원 확인 후 이 항목에 채워야 함

모델 배정 기준: 품질이 중요한 생성(페르소나/단서/이벤트/엔딩)은 Claude Opus 5, 플레이어가 매 턴 응답을 기다리는 실시간 대화만 응답 속도를 위해 더 빠른 Claude Sonnet 5를 사용. 둘 다 환경변수(`LLM_MODEL` , `LLM_CHAT_MODEL`)로 교체 가능하게 구성.

3. 아키텍처

`backend` → (내부 API 키 인증) → `llm-proxy` → Anthropic Claude API

- NPC 페르소나 생성 LLM과 대화용 LLM을 역할별로 분리 — 페르소나는 세션 시작 시 NPC별 1회만 생성해 DB에 캐시하고, 이후 모든 대화 요청은 이 캐시된 페르소나를 재료로 사용
- `/internal/llm/**` 엔드포인트는 `X-Internal-API-Key` 헤더 검증(타이밍 공격 방지용 `MessageDigest.isEqual`)으로 보호 — 포트 접근만으로 제3자가 비용을 발생시킬 수 없도록 차단

4. LLM 활용 기능 (8종)

#	기능	모델	설계 포인트
1	NPC 페르소나 생성	Opus	Structured Output(JSON Schema)으로 배경 서사·예시 대사 확장. 범인인 경우 동기(초안)를 서사에 자연스럽게 엮되 노골적으로 암시하지 않도록 지시
2	실시간 대화	Sonnet	페르소나+대화이력+호감도+목적감+최근 마을 이벤트를 조합해 시스템 프롬프트 동적 구성. 프롬프트 캐싱(<code>cache_control</code>)으로 페르소나 불록 재사용, thinking 비활성화로 지연 최소화. 응답과 호감도 변화량(-5~+5)을 함께 Structured Output으로 산출
3	밤 사보타주 아침 알림 문구	Opus	장소·유형·대상만 전달, 범인을 특정할 정보(외형 등)는 아예 미포함
4	오답 고발 후 랜덤 이벤트 연출	Opus	실제 범인 정보를 메서드에 전달하지 않아 응답이 범인을 암시할 수 없도록 원천 차단
5	선물 반응 대사	Opus	캐릭터 성격·말투·말버릇 기반 1인칭 짧은 반응
6	오답 지목 시 억울함 반응	Opus	지목된 NPC는 오답 판정 시점상 항상 무고 — 실제 범인 정보 자체를 알지 못하는 것이 전제이므로 전달하지 않음
7	엔딩 스토리 생성	Opus	이름·성격·말투·동기·사보타주 유형을 받아 1인칭 자백 톤 4~6문장 생성. 세션당 1회만 생성해 DB에 캐시(재조회 시 재호출 없음)
8	단서 카드 생성 / 돋보기 명확화	Opus	NPC 스프라이트 생성용 외형 묘사(<code>appearanceDesc</code>)를 그대로 재사용해 5주제(머리카락/소지품/발자국/혈흔/자국)별 가이드로 문구 생성. 이름·정체 언급 금지를 프롬프트에 명시

5. 프롬프트 엔지니어링 설계 원칙

- **스포일러 방지** — 이벤트·단서·오답반응 생성 시 실제 범인의 이름·정체를 프롬프트에 아예 넣지 않는 방식으로, "실수로 언급하지 말라"는 지시가 아니라 정보 자체를 주지 않는 구조적 차단을 우선함
- **캐릭터 일관성 + 비용 절감** — 같은 세션 안에서 변하지 않는 "캐릭터 시트"(이름/성격/말투/배경)를 별도 시스템 블록으로 분리해 프롬프트 캐싱 브레이크포인트로 고정, 틴마다 바뀌는 정보(호감도·목격담 등)는 뒤에 별도로 붙여 캐시 적중을 깨지 않게 함
- **호감도 기반 톤 분기** — 70점 이상/30~70점/30점 미만 3구간에 각각 다른 태도 지시문을 시스템 프롬프트에 삽입 (우호적으로 단서 제공 시도 / 예의는 지키되 단서는 먼저 안 꺼냄 / 경계·회피하며 완전히 솔직하지 않은 태도)
- **구조화 출력** — JSON Schema 기반 Structured Output으로 파싱 실패 위험을 줄이고, 대화 응답과 호감도 변화량을 한 번의 호출로 함께 받음 (숫자 범위 제약은 스키마로 강제할 수 없어 애플리케이션 코드에서 방어적으로 clamp)
- **7일차 이후 대화 제한** — 추리 대화가 금지되는 구간에서도 대화 자체는 막지 않고, "사건 관련 질문은 캐릭터답게 얼버무리고 새 정보는 주지 않는다"는 지시로 자연스럽게 제한

6. 안전 · 신뢰성 장치

- **거부 응답(refusal) 감지** — Claude가 안전 정책상 응답을 거부하면 `stop_reason: REFUSAL` 로 응답. 이를 감지해 명확한 예외로 걸러 기존 폴백 로직으로 연결. (개발 중 `Optional<StopReason>` 을 문자열로 잘못 비교해 이 감지가 무력화되어 있던 버그를 테스트 작성 과정에서 발견·수정)
- **전 구간 폴백 콘텐츠** — 페르소나/대화/단서/이벤트/엔딩 등 모든 LLM 호출 지점에 정적 대체 문구를 준비해, LLM 장애 시에도 게임 진행이 끊기지 않도록 설계
- **타임아웃 여유 확보** — backend↔llm-proxy 간 read timeout을 20초 → 45초로 상향. 실측 지연(페르소나 생성 ~23초, 랜덤 이벤트·엔딩 ~20~24초)이 기존 20초 한도에 걸쳐 있던 것을 실제로 재현해 확인 후 조정

7. 비용 / 호출 관리

- 프롬프트 캐싱으로 같은 NPC와의 반복 대화 시 입력 토큰 일부를 캐시로 대체 (사용량 로그에 `cacheRead` 값으로 적중 여부 확인 가능)
- 계정별 분당 호출 제한(슬라이딩 윈도우, 기본 20회/60초) — 대화 API에 적용
- 고발(고발→랜덤 이벤트 생성) API 하루 1회 제한 — 이전에는 같은 날 오답을 반복할 때마다 무제한으로 LLM이 재호출되는 구멍이 있었음을 점검 중 발견해 차단
- 페르소나·엔딩 스토리는 세션당 1회만 생성 후 DB 컬럼에 캐시, 이후 조치는 LLM 재호출 없이 즉시 반환

8. 검증 방식

모든 LLM 연동 기능은 실제 Aiven(MySQL) DB와 실제 Claude API 호출로 라이브 검증했다. 예를 들어 단서 카드 생성은 실제 세션(범인: 명자유)에서 "어깨 아래까지 오는 길고 곧은 검은 머리"라는 외형 묘사가 실제 생성된 단서 문구와 일치하면서도 범인 이름은 노출되지 않음을 확인했다. 그 외에도 `LlmServiceTest` (성공 경로 + LLM 예외 시 폴백 경로), `InternalApiKeyFilterTest` 등 단위 테스트로 폴백/인증 로직을 회귀 검증하고 있다.

9. 한계 및 향후 계획

- 코드 작성 자체에 사용한 AI 코딩 도구명(2번 항목)을 팀 확인 후 채워야 함
- 현재 단일 인스턴스 전체의 인메모리 레이트리밋 — 서버 다중화 시 Redis 등으로 교체 필요(이번 규모에서는 보류)
- 호감도 70점 이상 구간의 우호적 대화 톤은 라이브 세션에서 실제로 도달해 확인하지는 못함(코드 경로는 다른 구간과 동일하게 검증됨)